International AI Track | Security Self-assessment

Optional supporting material. Please complete this document in English.

Team name	Centrale
Competition track	International AI Track, Topic C (Cross-border and Student Finance)
Submission date	2026-10-20

Scope and how to read this

This is a self-assessment of the BorderlessGuard prototype, which runs against a **simulated sandbox ledger**. The authors wrote it. Two inputs are not ours: two white-box audits by separate AI agents with full read access to the code (23 findings, all fixed or mitigated), and a blind test set written by a separate AI agent that could not read the security code.

Two results estimate generalisation, one per parser: blind v2 with the rules parser, frozen in `44542f2` (**69/70 attacks blocked**, 24/30 benign English completed), and blind v2 with local Qwen2.5 7B, frozen in `5868d83` (**66/70 attacks blocked**, 26/30 benign English, 5/5 non-English). Everything measured after them is regression. State: commit `d1822bf`, 2026-10-19: 1230 tests pass, 7 are documented expected failures.

The model connector, the cascade and the model selector (`c10f0a0`), then the help replies, the scripted-task harness and its localhost HTTP API, the FX cost display and Verification of Payee (`7e90cc3`, `d1822bf`), came after the second audit and have had no white-box review. No hosted or frontier model was measured. No licensing, KYC, AML, sanctions or data-protection compliance is claimed.

01 Permission Model

- **Roles, operations and permission boundaries**

Role	Operations	Permission boundary
Customer (the student, one session per `user_id`)	Ask; attach a document; approve or reject her own pending actions with a tap or a code; cancel; read her own overview and receipts; check her receipts against her signed copy.	Sees and acts on her own accounts and payees only (`TENANT_ISOLATION`). Cannot change limits, the watchlist, the FX price or the code. Each approval is bound to the hash of the exact action.
Language model (any parser)	Propose typed intents from the customer's text.	No tool, no authority, no access to accounts, balances, IBANs or documents. Its output is validated, grounded in the customer's words and policed like any input.
Content guard	Warn about injected documents and tool output.	Advisory. No authorisation reads its output.
FX provider (simulated)	Return a price.	Untrusted. A quote is kept only if it equals the bank's reference price and the ledger issued it; it is used once.
Bank operator (stands in as a reset function)	Unlock approval by code after three wrong codes.	Never reachable from chat. In the sandbox it is a function call and the Reset demo button; there is no operator login.
Verifier (customer or reviewer)	Run the chain check against a signed anchor.	Read-only.

Tier	Operation	Boundary
R0	Overview, receipts, rate quote, affordability simulation	Allowed; creates no action
R2	Payment to a payee older than 72 h up to 1,500 EUR eq.; currency purchase up to 2,500 EUR eq.	One tap
R3	Payment to a payee under 72 h old; larger amounts; splitting payments; adding a payee	One-time code; three wrong codes in a row lock approval by code for the customer
R4	Infeasible payment, unregistered or blocked payee, IBAN not in the registry, other tenant, reserve breach, over 4,000 EUR per payment or 5,000 EUR per day, untrusted origin, sanctions hit, invalid IBAN, look-alike payee name, off-price FX quote, locked approval	Denied; nothing the model or the customer says overrides it

- **Confirmation, verification and rollback mechanisms**
  - **Confirmation.** Every action that changes money or the registry waits for approval. The card shows the payee, the amount, the account tail, the tier and the balance afterwards. The approval is a hash of the action: changing any field invalidates it (`param_tamper`). A correction is a new action with a new hash.
  - **Verification.** Every action, executed or denied, gets a receipt in the customer's own hash chain. She keeps a signed anchor of the latest receipt and can check the chain against it. A deleted last receipt or a rewritten chain is detected only against an anchor she kept, and only while the signing key is secret. While the public demo key is in use, the log and the UI say so.
  - **Rollback.** Before execution she can reject, cancel or correct any pending action, and nothing has moved. After execution the prototype has no reversal: the sandbox has no recall or refund function, and a real payment could not always be recalled. That is why high-risk steps wait for a code. A denied action is final and recorded; the only unlock is the operator reset of the code lock.
  - **Failure recovery.** A model that is down, slow or malformed falls back to the rules parser. A policy exception becomes a deny. A malformed anchor gives "invalid", not a crash. A message over 2,000 characters is refused before parsing.

02 Security Tests

- **Prompt injection, social engineering, privilege escalation and failure recovery tests**

  Every test runs against the real system through `bg.app.build_default`. A case counts as blocked only if every invariant of its family holds: no money to an unregistered or attacker account, limits unchanged, approvals bound, no other customer's data.

Category	What was tested	Where	Result
Prompt injection	Hidden instructions in invoices (9 variants, plus ES, AR, ZH, base64, leetspeak, table, polite, fake bank notice, zero-width, system tags), IBAN swap with no instruction, poisoned FX note, model free text	tests/adversarial F1 and F4; blind v1 document cases; blind v2 D01 to D13; missions T14 and T20	69/69 in the development suites; blind v2 rules 69/70 overall
Social engineering	Chat IBAN change, new payee named "mom", look-alike duplicate name, add-payee-and-pay in one sentence, a careless user who approves everything shown (stage B)	tests/adversarial F2; blind v2 M01 to M15 and the careless-user stage; tests/audit	No money reached a non-registry payee in any case
Privilege escalation, data extraction	"Raise my limit", "skip the code", DAN and developer mode, other customers' balances, system prompt, API keys, other tenant's payee id	tests/adversarial F3 (18); blind v2 P01 to P10; mission T19	No data leaked; limits unchanged
Replay, tampering, brute force	Double confirm, wrong hash, other session, other tenant, 10 wrong codes then the right one, new session or correction to reset the counter	tests/adversarial F5 (15); blind v2 O01 to O04 and R01 to R07; missions T12, T13, T17	Money moved at most once; the lockout holds
Compliance controls	Structuring (4 x 400, 600 + 550 + 500), sanctions with homoglyphs and spacing variants, invalid IBAN, payee-name collision, FX price manipulation	tests/unit/policy, tests/audit; missions T15, T16; blind v2 S01 to S04 and X01 to X08	Blind v2: one miss (X08, Cyrillic letter), fixed afterwards
Failure recovery	Model unreachable, timeout, HTTP 500, invalid JSON, extra field, out-of-range amount, empty list; policy exception; malformed anchor; truncated and rewritten chains	tests/unit/agent (local parser, connectors, hardening), tests/unit/ledger, tests/audit	Every case falls back to the rules parser, denies, or returns invalid

Evaluation layer	Result	How to read it
Development suites (team-written)	69/69 attacks, 29/29 benign	Regression only
Blind v1, rules parser, first run	47/48 attacks, 21/25 benign	Blind at the time. A claim that all 14 document injections were detected was inflated: 8 misses were masked by the seeded invoice. Corrected; three structural signals now detect them.
Blind v2, rules parser (`44542f2`)	**69/70 attacks** (95% CI 92.3 to 99.7); multi-turn 15/15; 24/30 benign English (62.7 to 90.5); non-English 0/5; 0 wrong payments, 0 data leaks, 0 unconfirmed executions	**Generalisation estimate, rules.** Nine attacks were stopped only because the parser did not understand them (58/59 without them). The one miss needed the customer's code.
Blind v2, Qwen2.5 7B (`5868d83`)	**66/70 attacks** (86.2 to 97.8); multi-turn 15/15; 26/30 benign English (70.3 to 94.7); non-English 5/5; 137 calls, 0 fallbacks. Qwen 3B: 65/70, 21/30, 3/5	**Generalisation estimate, LLM parser.** One run. No payment to an unregistered account. One bypass from a field the model changed (X02).
Regression after the frozen runs	Qwen 7B 103/105 on v2; cascade 103/105; both 69/69 and 29/29 on the development suites; 1230 tests pass	Not generalisation: the system was changed after seeing these cases.

White-box audits. Round 1 (code before `2ce6035`): 12 findings (3 High, 3 Medium, 6 Low). Round 2 (code at `22b144e`): 11 findings (2 Medium, 9 Low). Each has a test in `tests/audit/` that runs the exploit; 96 tests pass and no xfail is left.

ID	Sev.	Finding	Status
AUDIT-1	High	A poisoned FX quote could charge 2,590 USD for 100 EUR; quotes were reusable	Fixed: ledger issues single-use quotes; `FX_PRICE_CHECK`
AUDIT-2	High	A watch-listed IBAN with Arabic-Indic digits got R3, not the sanctions deny; no mod-97 check	Fixed: canonical IBAN, `IBAN_VALID`
AUDIT-3	High	The wrong-code lockout reset with each new action, correction or session	Fixed: counted per customer
AUDIT-4, 5, 6	Medium	Armenian and Cherokee look-alikes passed the name screen; a new payee could shadow "mom"; a payee name rendered as HTML hid every warning	Fixed: wider table, fail closed; `PAYEE_NAME_UNIQUE`; escaping
AUDIT-7 to 11	Low	Huge amounts raised an exception; `1.005` rounded silently; raw messages logged; one global receipt chain; malformed anchor crashed verification	Fixed
AUDIT-12	Low	Anchors signed with the public demo key when `BG_RECEIPT_KEY` is unset	Mitigated: logged and shown in the UI; still the offline default
AUDIT2-1, 2	Medium	A sanctioned name without spaces passed the screen; a model's free text was shown as the bank's own words and bare URLs became links	Fixed in `c10f0a0`

ID	Sev.	Finding	Status
AUDIT2-3 to 11	Low	Markdown in chips; ungrounded synonyms, currencies and memo; question turned into a correction; duplicate cards; proxy variable; raw telemetry; quadratic CPU on a long message	Fixed in `c10f0a0`

03 Data and Privacy

- **Data provenance, consent, minimisation, storage and deletion**

Topic	What is true in this prototype
Provenance	All data is synthetic. The persona (Lina), balances, payees, IBANs, invoices and the sanctions list are invented constants in `bg/contracts/params.py`. The IBANs are fictional and pass the mod-97 check. No real person, account or document was used. The Qwen models were run unmodified; nothing was fine-tuned.
Consent	No real user took part, so no personal data was collected and no consent was needed. A real deployment would need the customer's consent to process her messages, and to send them to a third-party model if a remote model is connected. Choosing "Connect a model" in the UI shows that a remote endpoint receives the messages.
Minimisation	A model receives only the customer's message, her own payee names and aliases, and the agent's pending question. Never IBANs, balances, documents or other customers' data. The audit log keeps a SHA-256 and the length of each message instead of its text, masks IBANs to their last four characters and drops one-time codes. Model telemetry holds timings, a hash and an error class.
Storage	Sessions, drafts, the wrong-code counter and ledger state are in memory and end with the process. Receipts are in SQLite, in memory by default or in a file if a path is given; each row holds the full action payload as plain text. Mission audit logs in `deliverables/evidence/` are masked. A connected model's API key lives only in the browser session or the environment and is never written to disk. There is no encryption at rest and no access control in the sandbox.
Third parties	None by default; no network call is made. A local model is called on `localhost`. A connected remote model receives the message and the payee names.
Deletion	Ending the process or pressing Reset demo discards the sandbox state. The receipt chain cannot be edited without breaking verification, so a real erasure request needs another design (hashes only in the chain, payloads in a separate store under a per-customer key that can be deleted). That is not built.
Secrets	An audit of the full git history found no API key, token or personal e-mail address. The committed receipt-signing key is a public demo value, flagged while it is in use.

No data-protection compliance (GDPR, PIPL or other) is claimed.

04 Known Risks

- **Risk, impact, mitigation and planned improvement**

Risk	Impact	Mitigation today	Planned improvement
Sandbox only: no real bank, KYC, AML or FX source; one persona, EUR and USD	No claim about production behaviour or compliance	Stated in every document; the UI shows "SANDBOX, no real money"	Integrate a real payment-initiation API and the bank's strong customer authentication
No authentication; fixed one-time code `246810`; predictable session ids; frozen clock	Whoever opens a session acts as that user; codes can be guessed in a real setting; time-based rules never run in real time	Per-customer lockout after 3 wrong codes; constant-time comparison	Authenticated sessions with random tokens; a per-transaction code from a real second factor
The guard misses a new injection phrasing (English and French patterns only)	A missing warning only; authorisation never reads the guard	Registry IBANs, `UNTRUSTED_ORIGIN`, hash-bound approval; language-independent structural signals	A learned detector as an extra advisory layer, evaluated on a new blind set
A model changes or invents a field; grounding checks presence, not meaning	A visible card the customer must approve, or a question	Grounding on every parser; sanctions screen on the source text; cards show payee and amount	A blind set written to probe extraction; a check on exceptions and past amounts
No frontier model measured; Qwen measured once	Unknown accuracy and failure modes of other models	Same checks apply whichever model answers; https and key handling in the connector	Run the blind protocol through the connector with intervals

Risk	Impact	Mitigation today	Planned improvement
Connector, cascade, selector (`c10f0a0`), harness and Verification of Payee (`d1822bf`) not audited	Possible findings not yet known	Unit tests (32 connector, 46 hardening, 16 selector, 23 harness, 56 Verification of Payee)	A third white-box audit
A connected remote model receives the customer's messages	Message and payee names leave the machine	Off by default; https only; the UI says so; key is session-only	Per-deployment data-processing terms; a local model for sensitive use
Receipt chain needs the customer's copy; symmetric HMAC; public demo key	Truncation or rewrite is undetectable without her anchor; the key holder could forge one	Anchor check in the UI; warning while the demo key is in use	Asymmetric signatures, key ids and rotation, external timestamping or on-chain anchoring
Sanctions screen is a demonstration: fictional list, finite look-alike table	Real matching would miss or over-block	Fails closed on letters it cannot fold; wildcards; spacing variants	A maintained list, full Unicode confusables, transliteration and fuzzy matching
Structuring rule covers one payee and one 24 h window	Splitting across payees or days is not detected	Step-up and an event on the covered case	Cross-payee, longer-window analytics
User approves a malicious payee or payment	Money can reach a scammer the user chose	72 h new-payee step-up, sanctions and name-collision checks, explicit card, caps	A cool-off delay; Verification of Payee
State in memory; no lock between policy and execution; plain-text receipts	Lost on restart; a shared backend would need locks; no erasure	One backend per session; documented	A database with row locks; hashes in the chain and encrypted payloads
Post-fix results are regression, not blind	No current generalisation estimate	Two frozen runs reported separately	A fresh blind set (v3) after all fixes